Retrieval-Augmented Generation

Baseline Retrieval vs. Cross-Encoder Re-ranking

D. Subhash | Roll No. 2025201007

github.com/SubhashChandraDangeti/RAG-Mini-Project

Experimental setup

The experiment compared two RAG pipelines to test whether re-ranking improves the evidence supplied to an LLM. The corpus was the *US Chess Federation's Official Rules of Chess, 7th Edition*, free-download version revised on 21 August 2020, containing Chapters 1, 2, 10, and 11. Its general rules, exceptions, and separate online-play provisions create useful retrieval distractors. Answers were evaluated against this document edition.

Corpus source: Official US Chess rulebook PDF (free-download edition). Save the downloaded file as `US_Chess_RuleBook.pdf` to reproduce the notebook experiment.

Component	Configuration
Corpus processing	139 PDF pages; 126 retained after cleaning; 405,752 characters.
Chunking	190-word windows; 40-word overlap; page-local chunks; 484 chunks total.
Metadata	Stable chunk IDs and original PDF page numbers.
Bi-Encoder	`sentence-transformers/all-MiniLM-L6-v2`
Vector index	NumPy matrix of 484 normalized vectors, each with 384 dimensions.
Cross-Encoder	`cross-encoder/ms-marco-MiniLM-L-6-v2`
Answer model	`gemma-4-26b-a4b-it`, Google GenAI client; temperature 0.0.
Retrieval sizes	Top 10 initial candidates; Top 3 context chunks for both pipelines.

PyMuPDF extracted text from PDF page 13 onward, excluding the cover, update summaries, and table of contents. Cleaning removed repeated headers/footers, isolated page numbers, and line-break hyphenation; pages with fewer than 150 cleaned characters were excluded. Both pipelines used identical chunks, initial candidates, prompt structure, LLM, temperature, and context count. The prompt requested evidence citations and an admission when context was insufficient. Only the final context selection and order changed.

1. Architecture comparison

Bi-Encoder. Query and chunk text are embedded independently. Document vectors are precomputed once; at query time, a query vector is compared with them using cosine similarity:

$$\mathrm{sim}(q, d) = \frac{\mathbf{q}^{\mathsf{T}}\mathbf{d}}{\|\mathbf{q}\|_2\|\mathbf{d}\|_2}.$$

Because the implementation normalizes vectors, their dot product gives cosine similarity directly. The baseline retrieves ten chunks and supplies the first three to the LLM. This is fast, but topical similarity can miss an exception or a precise timing condition.

Cross-Encoder. Each query and candidate chunk are processed together, with joint self-attention across their tokens, to produce a relevance score. All ten retrieved pairs are scored, sorted in descending order, and the new Top 3 are passed to the same LLM prompt. This enables finer relevance judgments at an additional inference cost. Cross-Encoder scores and cosine scores have different scales; only their rankings are compared.

2. Qualitative and rank-shift analysis

Eight queries tested exceptions, negation, constraints, and document-specific corner cases. Reference evidence was identified using phrase pairs from the PDF. Two cases placed the critical evidence outside the baseline Top 3 but inside the Top 10, allowing re-ranking to recover it for generation.

Query	Reference chunk	PDF page	Baseline	Re-ranked
Q3	P040-C01	40	10	1
Q6	P022-C01	22	4	1

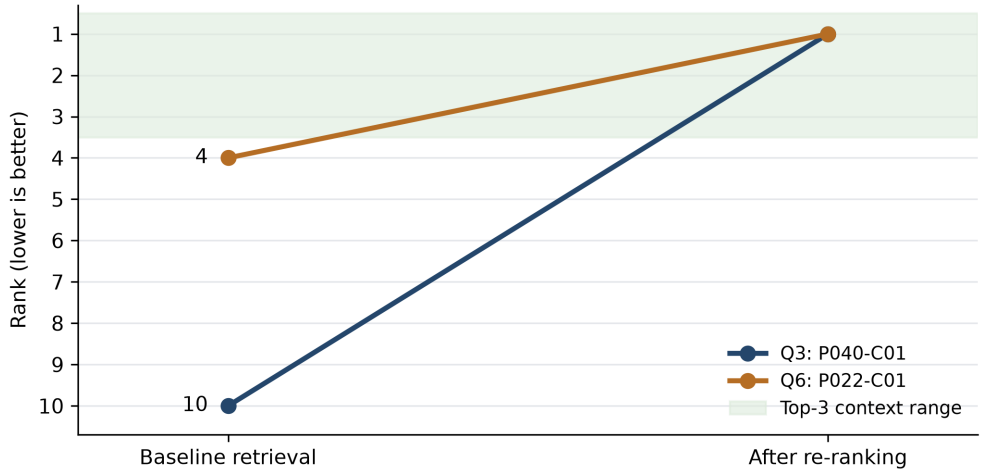


Figure 1. Recorded rank changes for the two critical evidence chunks. The shaded region is the Top-3 context range.

2.1 Q3: Checkmate and expired clocks

Query: "Does checkmate count when both players have no time left?"

Pipeline	Context chunk IDs in rank order
Baseline	P031-C01, P034-C02, P026-C01
Re-ranked	P040-C01, P034-C03, P031-C01

Answer 1: The baseline explained that checkmate takes priority over a subsequent flag fall and immediately ends the game without requiring the clock to be stopped. It also discussed a time-forfeit claim preceding checkmate. However, it omitted the specific both-flags-down condition in sudden death.

Answer 2: After re-ranking, the answer included the missing exception: if a player points out that both flags are down *before checkmate*, the game is drawn in sudden death. It also discussed director discretion and the distinction involving increments of 30 seconds or more.

Analysis: The decisive chunk moved from rank 10 → 1. The baseline supplied general checkmate rules, whereas re-ranking supplied the passage addressing this corner case. The observed baseline failure was omission of an essential condition, not an invented rule. Generation was still imperfect: Rule 14G1 concerns the timing of the *both-flags-down claim* relative to checkmate, while Answer 2's unclear-timing explanation used "flag fall." Improved retrieval therefore did not guarantee perfectly precise wording.

2.2 Q6: Replacing a clock during a game

Query: "After a chess game has started, can its clock ever be replaced, and how must the replacement be configured?"

Pipeline	Context chunk IDs in rank order
Baseline	P129-C02, P121-C03, P029-C03
Re-ranked	P022-C01, P121-C03, P049-C04

Answer 1: "The provided context is insufficient." The baseline did not have the replacement rule among its three context chunks and appropriately abstained.

Answer 2: The re-ranked answer explained that a defective clock may be replaced, referenced the exceptions in rules 5F5 and 14H2a, and stated that a replacement for a defective clock must be set in the same manner as the clock being replaced.

Analysis: The supporting chunk moved from rank $4 \rightarrow 1$, changing an unanswered question into a supported answer. This was a retrieval/context failure rather than hallucination. The evidence already existed in the ten candidates; the Cross-Encoder promoted it past the three-chunk cutoff.

2.3 Other observations and limitations

Q1, Q4, Q5, and Q8 already had reference evidence at rank 1 and retained that best rank. Q7's reference moved from rank 1 to 3. Q2's designated reference was absent from the Top 10, so re-ranking could not recover it. Phrase matching is not exhaustive relevance annotation. The queries were deliberately stress-tested and Q3 was refined; the two successes illustrate failure modes rather than an unbiased overall accuracy estimate.

3. Latency profiling

Each query received one warm-up followed by five timed repetitions using `time.perf_counter()`. Baseline timing included query embedding and dense search. Re-ranking timing included Cross-Encoder scoring and sorting. Model loading, document embedding/index construction, and LLM generation were excluded.

The table reports component medians. The two-stage total is the *sum of the component medians*, rather than the median of total times.

Query	Baseline (ms)	Re-ranking (ms)	Total (ms)
Q1	22.20	429.04	451.24
Q2	17.63	432.63	450.26
Q3	15.11	358.72	373.84
Q4	20.37	389.59	409.96
Q5	16.48	382.99	399.46
Q6	16.78	382.18	398.97
Q7	17.34	373.43	390.76
Q8	21.11	383.95	405.06
Mean of medians	18.38	391.57	409.94

Values are independently rounded; displayed components can differ from the displayed total by 0.01 ms.

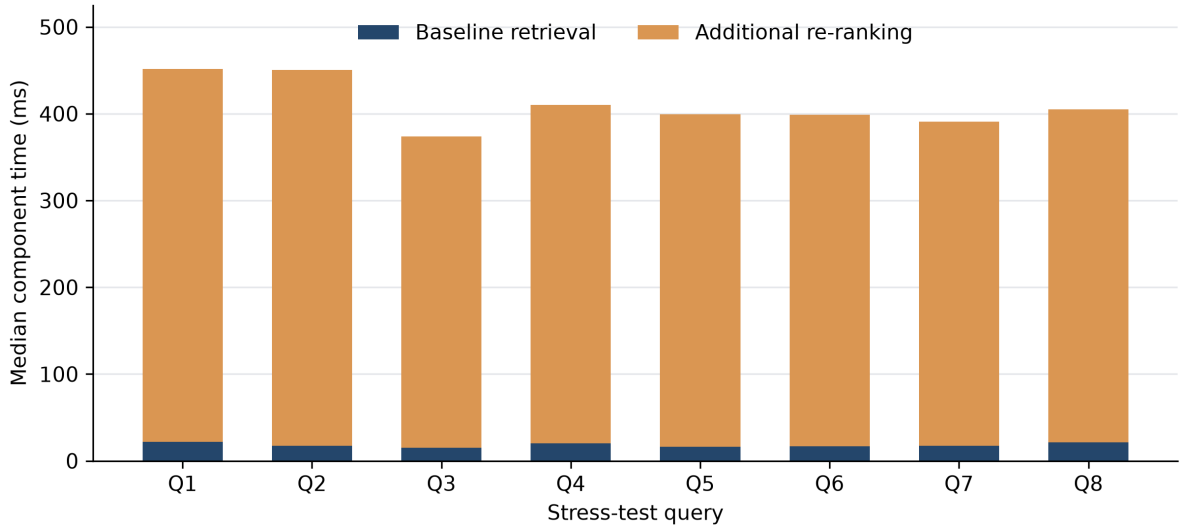


Figure 2. Retrieval-time breakdown. Each bar combines the baseline median and the additional re-ranking median; it excludes LLM generation.

Re-ranking added approximately 392 ms per query on average, increasing mean retrieval time from approximately 18 ms to 410 ms. The ratio of these rounded means is about 22.3. Cross-Encoder inference dominated retrieval time, but it introduced essential evidence into the final context for Q3 and Q6. This trade-off may be worthwhile where omitting a rule exception has a meaningful effect on the answer.

These are retrieval measurements, not end-to-end generation times. API overload retries affected generation separately. The saved run did not record CPU/GPU specifications or device selection, so these local Windows/Jupyter measurements should not be generalized across hardware.

4. Scaling: why not use a Cross-Encoder first?

A Cross-Encoder score depends on a particular query and passage together. It cannot be precomputed once per document and reused for unrelated questions. Scoring a collection of N chunks directly therefore requires N query–chunk evaluations for every question, even if pairs are batched.

For this corpus, scoring all 484 chunks would involve 48.4 times as many pairs as re-ranking ten candidates. This is a pair-count comparison, not a measured runtime multiplier: batching, input lengths, and hardware affect actual latency. At a million chunks, full scoring would require a million pairs per query.

The Bi-Encoder avoids repeated document encoding by storing chunk vectors. The current NumPy implementation performs exact search across those vectors; larger collections can use approximate nearest-neighbour indexes. Dense retrieval narrows the collection to ten candidates, and expensive joint scoring is reserved for that small set. Its limitation is candidate recall: evidence absent from the Top 10 remains unavailable to re-ranking, as observed for Q2's reference chunk.

Conclusion

Re-ranking promoted Q3's critical passage from rank 10 to 1 and Q6's from rank 4 to 1. This added a missing corner-case condition in Q3 and enabled a supported answer in Q6. The gains cost approximately 392 ms in additional retrieval time on average. The experiment supports combining fast dense candidate discovery with focused relevance scoring, while retaining the need to check candidate recall and the faithfulness of generated answers.